

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

[Github-Deep-Learning--Labs/Lab-3 at main · Harsha-2828/Deep-Learning--Labs](#)

1.Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

1. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
 - Activation Layer
 - Pooling Layer
 - Fully Connected Layer
- The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size P

Padding S

Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied elementwise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

```

  0 0
  0 0
  0 0
  1 1
  0 0

```

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

```

  -1 1
  -2 0 2
  -1 0 1

```

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

```

  -1 0 -1
  2 0
  1 1

```

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ 0 & 4 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ 0 & 5 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -2 & 1 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 8 & -1 \\ 0 & -1 & -1 \\ 0 & -1 & 0 \\ -1 & 0 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3 × 3	Preserve original image	Baseline comparison
Sobel-X	3 × 3	Detect vertical edges	Object detection
Sobel-Y	3 × 3	Detect horizontal edges	Boundary detection
Laplacian	3 × 3	Detect edges in all directions	Medical imaging
Sharpen	3 × 3	Enhance details	Image enhancement
Box Blur	3 × 3	Smooth image	Noise removal
Gaussian Blur	3 × 3	Reduce noise	Computer vision
Emboss	3 × 3	3D effect	Artistic filtering
Outline	3 × 3	Boundary extraction	Segmentation
Motion Blur	5 × 5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{array}{cccc} 1 & 5 & 2 & 3 \\ 2 & 3 & 1 & 8 \end{array} \quad \begin{array}{c} \left[\begin{array}{cccc} 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \end{array} \right] \end{array}$$

Using a 2×2 max pooling filter, Window 1

$$\begin{array}{cc} " & \# \\ 1 & 5 \\ 7 & 8 \end{array} \rightarrow 8$$

Window 2

$$\begin{array}{cc} " & \# \\ 2 & 3 \\ 1 & 0 \end{array} \rightarrow 3$$

Window 3

$$\begin{array}{cc} " & \# \\ 4 & 6 \\ 2 & 3 \end{array} \rightarrow 6$$

Window 4

$$\begin{array}{cc} " & \# \\ 9 & 5 \\ & \rightarrow 9 \\ 1 & 8 \end{array}$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$\begin{aligned} &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

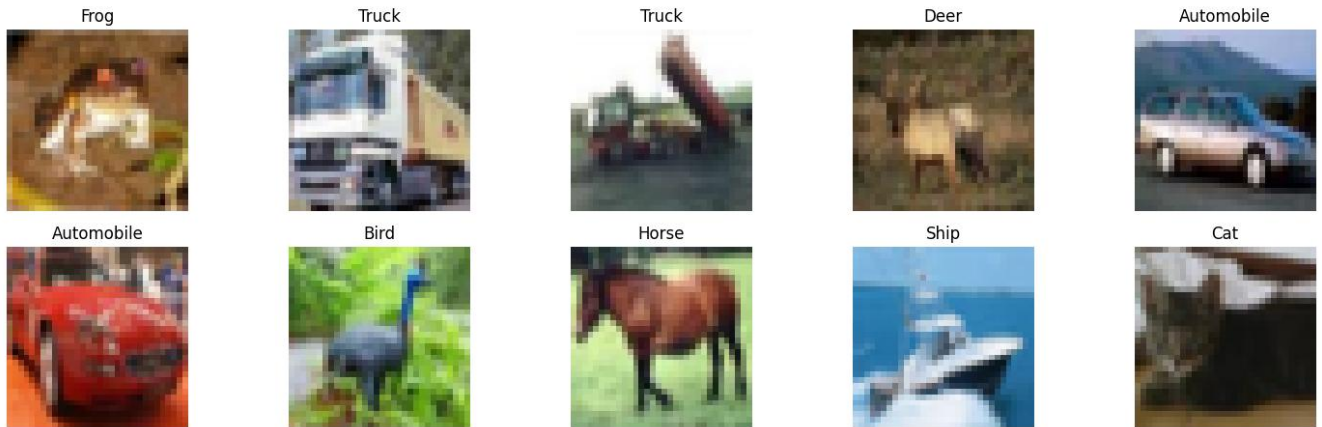
8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.

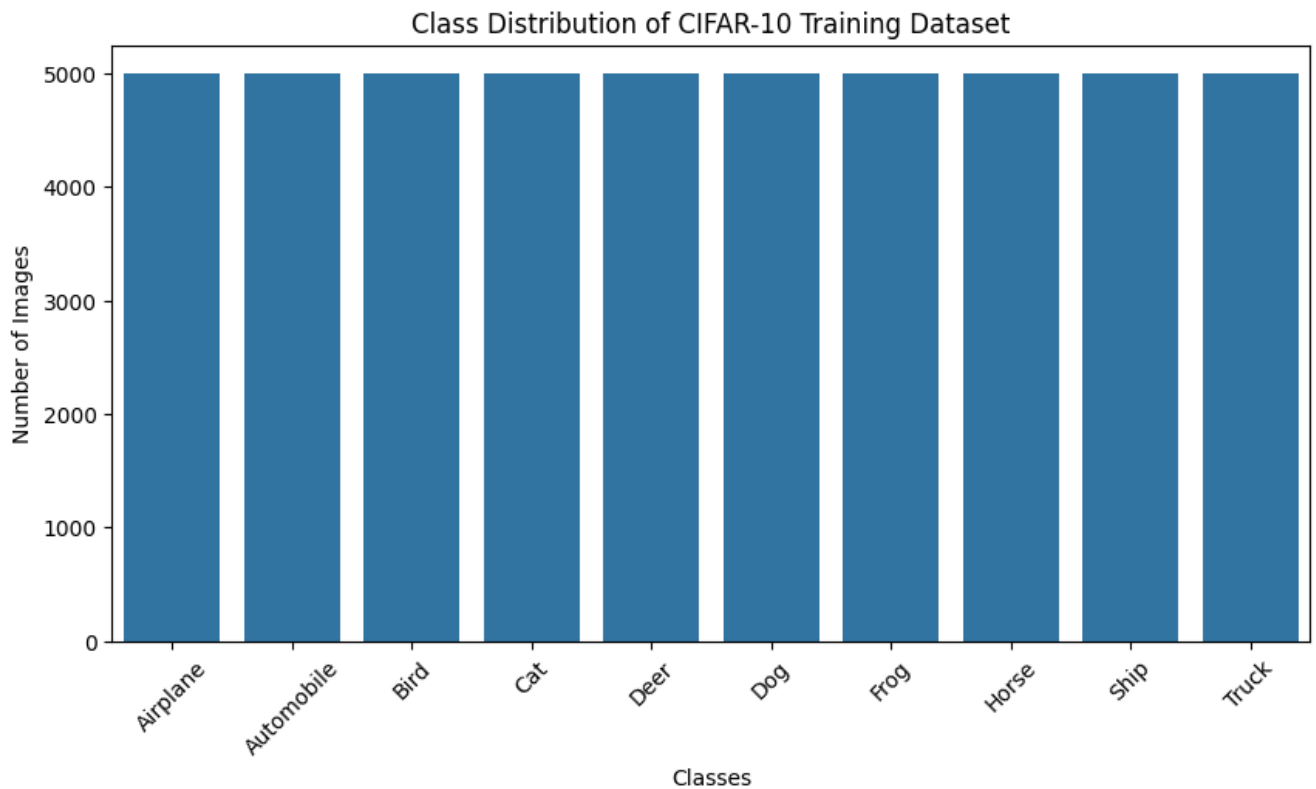
Sample Images from CIFAR-10 Dataset



- Print dataset dimensions.

```
Training Images : (50000, 32, 32, 3)
Training Labels : (50000, 1)
Testing Images  : (10000, 32, 32, 3)
Testing Labels  : (10000, 1)
```

- Plot class distribution.



Images per Class	
Airplane	: 5000
Automobile	: 5000
Bird	: 5000
Cat	: 5000
Deer	: 5000
Dog	: 5000
Frog	: 5000
Horse	: 5000
Ship	: 5000
Truck	: 5000

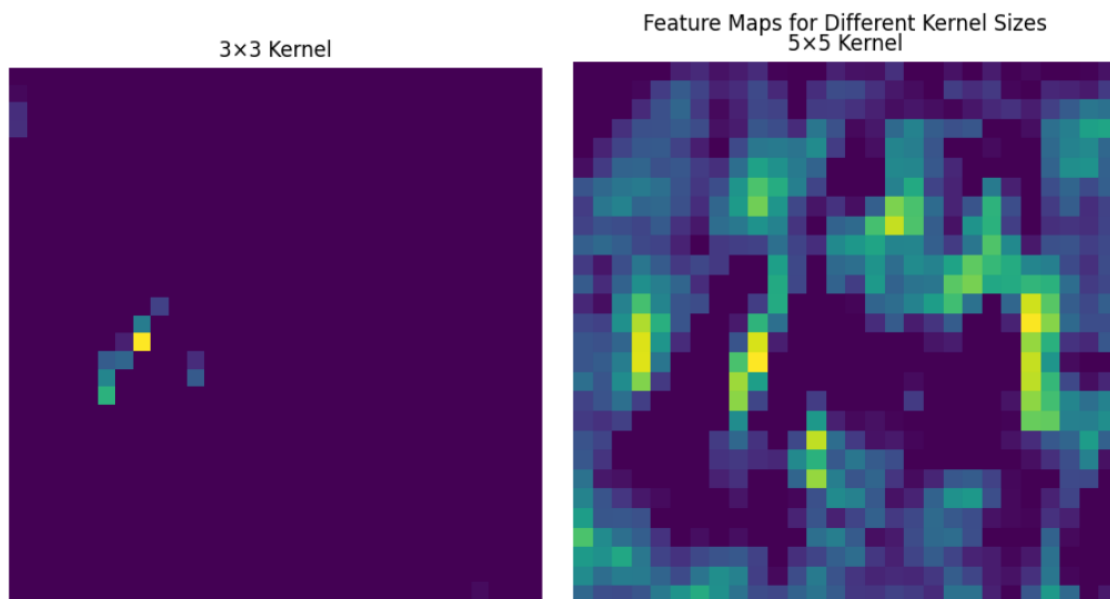
Task 2

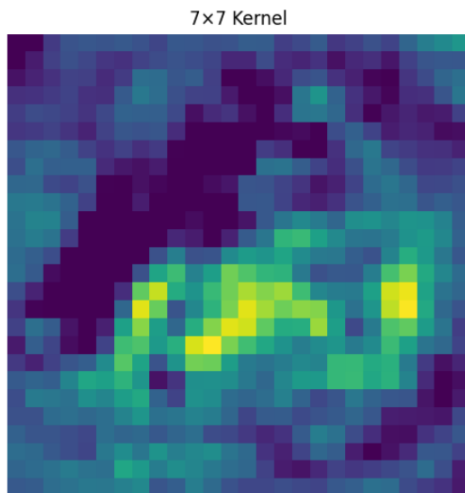
Implement a convolution layer. Compare

- Kernel = 3×3 • Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Feature Map Sizes	
Kernel 3×3	---> (1, 30, 30, 1)
Kernel 5×5	---> (1, 28, 28, 1)
Kernel 7×7	---> (1, 26, 26, 1)





Task 3

Study hyperparameters. Compare

- Stride = 1
- Stride = 2
- Padding = Same • Padding = Valid

Compute output dimensions.

```
Stride = 1 | Padding = VALID --> (1, 30, 30, 1)
Stride = 2 | Padding = VALID --> (1, 15, 15, 1)
Stride = 1 | Padding = SAME --> (1, 32, 32, 1)
Stride = 2 | Padding = SAME --> (1, 16, 16, 1)
```

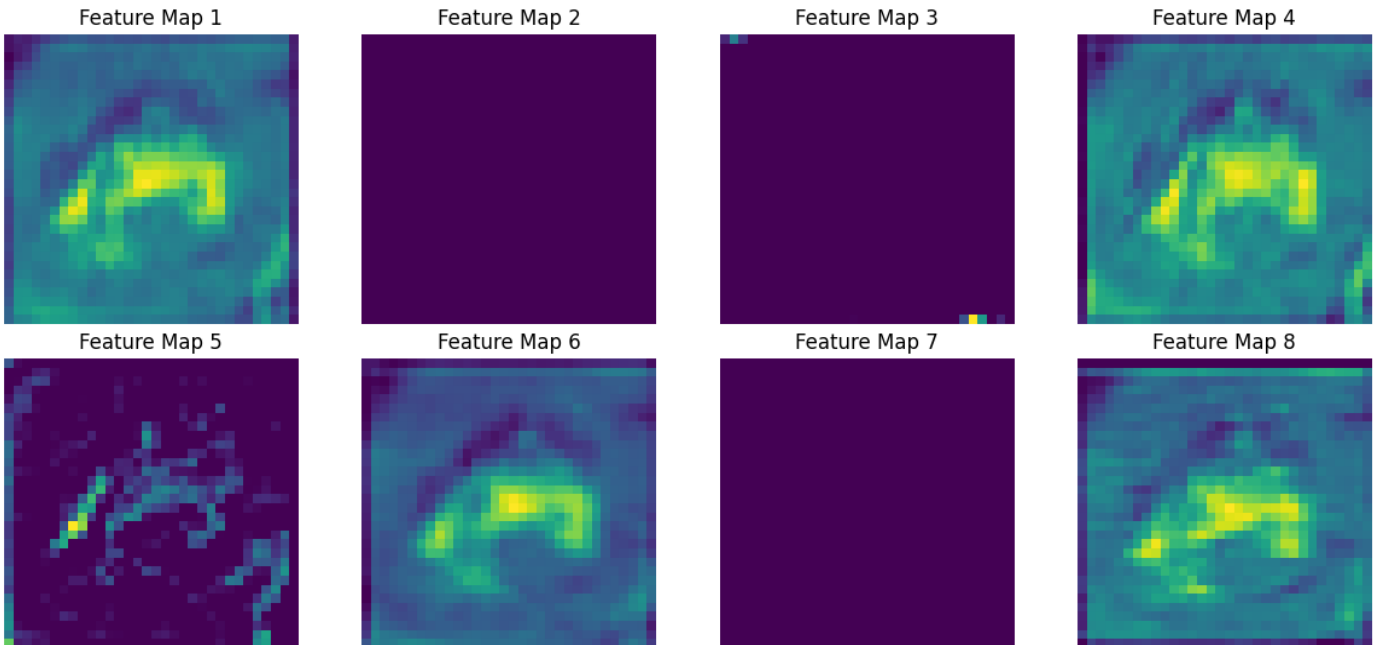
Output Dimension Calculation

```
Stride=1, Padding=Valid      : 30 x 30
Stride=2, Padding=Valid      : 15 x 15
Stride=1, Padding=Same       : 32 x 32
Stride=2, Padding=Same       : 16 x 16
```

Task 4

Visualize feature maps after the first convolution layer. Display at least eight feature maps.

Feature Maps after First Convolution Layer

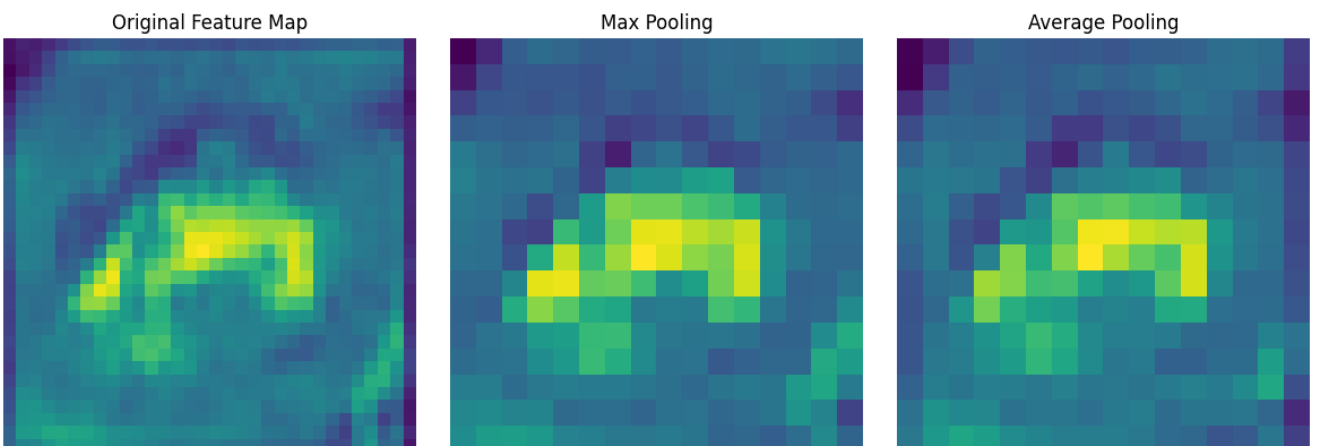


Task 5 Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

```
Input Shape      : (1, 32, 32, 1)
Max Pool Output Shape : (1, 16, 16, 1)
Avg Pool Output Shape : (1, 16, 16, 1)
```



Task 6

Construct the following CNN.

```
Kernel Size      : 3x3
Input Channels   : 3
Number of Filters: 16
Trainable Parameters = 448
```

Model: "sequential_11"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_11 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 128)	524,416
dense_1 (Dense)	(None, 10)	1,290

Total params: 545,098 (2.08 MB)

Trainable params: 545,098 (2.08 MB)

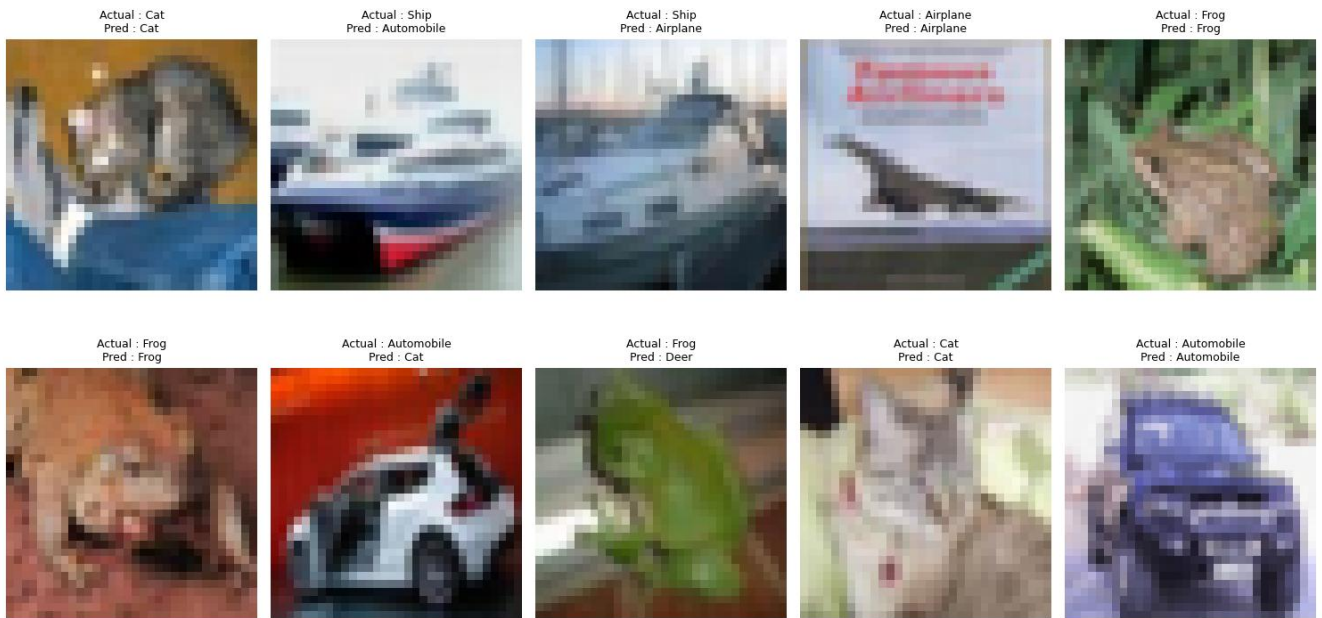
Non-trainable params: 0 (0.00 B)

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

- Test Loss : 2.1286
- Test Accuracy : 68.31%



Task 7

Evaluate

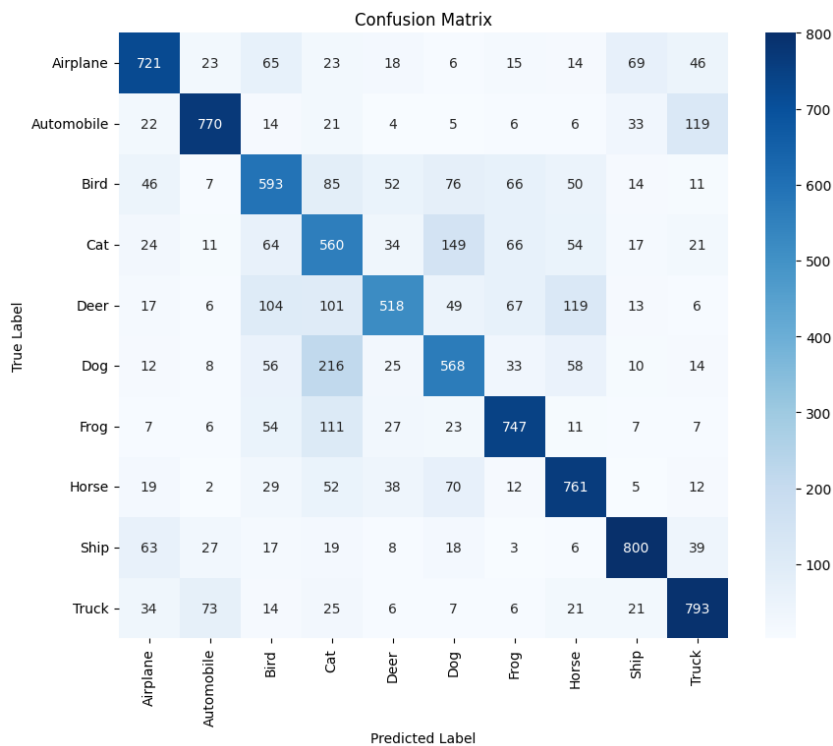
- Accuracy
- Precision
- Recall
- F1-score

```
Model Performance
-----
Accuracy : 0.6831
Precision: 0.6891
Recall   : 0.6831
F1 Score : 0.6837
```

- Confusion Matrix

	precision	recall	f1-score	support
Airplane	0.75	0.72	0.73	1000
Automobile	0.83	0.77	0.80	1000
Bird	0.59	0.59	0.59	1000
Cat	0.46	0.56	0.51	1000
Deer	0.71	0.52	0.60	1000
Dog	0.58	0.57	0.58	1000
Frog	0.73	0.75	0.74	1000
Horse	0.69	0.76	0.72	1000
Ship	0.81	0.80	0.80	1000
Truck	0.74	0.79	0.77	1000
accuracy			0.68	10000
macro avg	0.69	0.68	0.68	10000
weighted avg	0.69	0.68	0.68	10000

- Classification Report

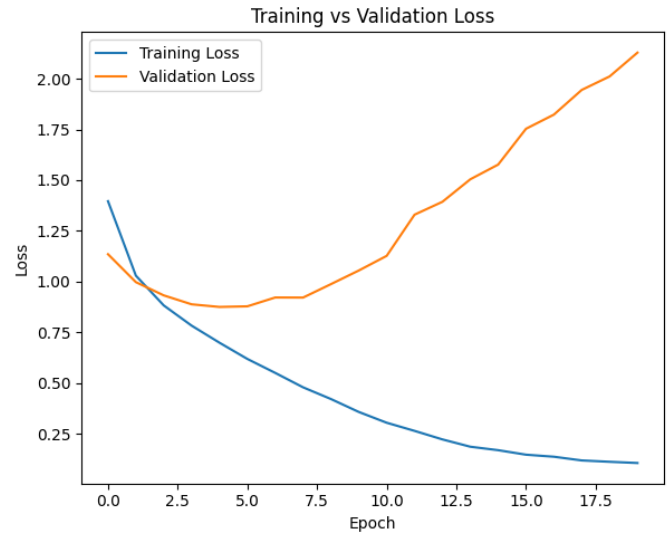
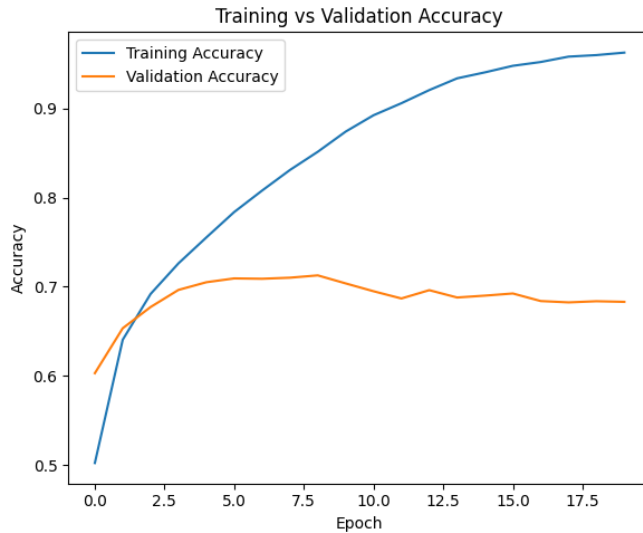


9. Mandatory Plots

Generate

1. Sample Images
2. Class Distribution
3. Training Accuracy

4. Validation Accuracy



5. Training Loss

6. Validation Loss

7. Feature Maps

8. Confusion Matrix

Write a 2–3 line inference for each plot.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

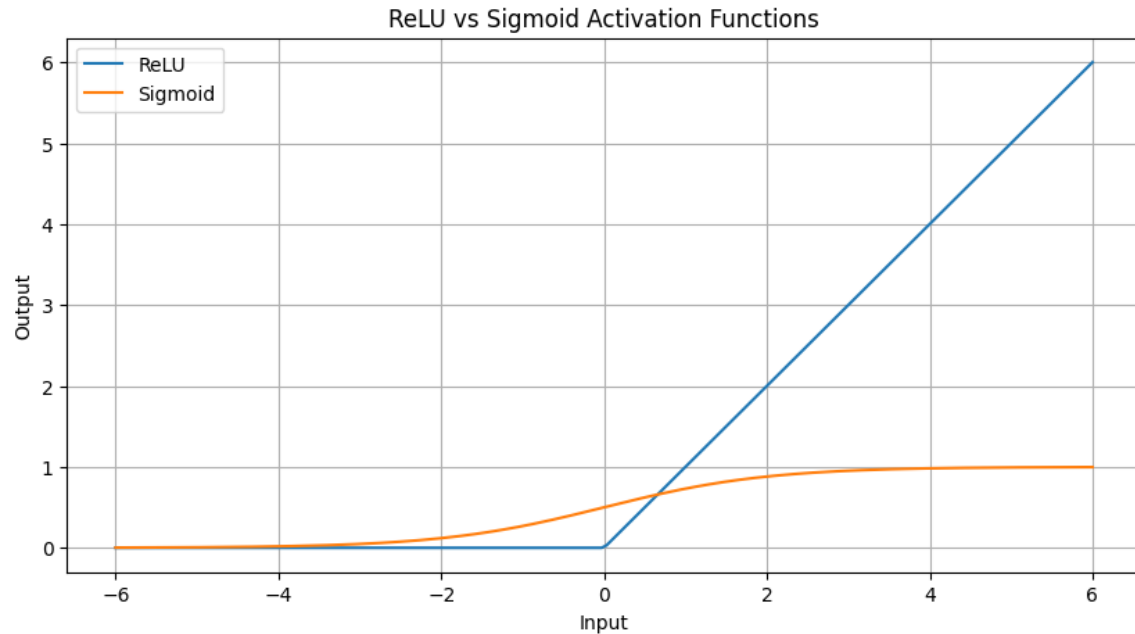
```
Input Size      : 64 x 64
Kernel Size     : 5 x 5
Stride          : 2
Padding         : 2

Output Size     : 32 x 32
```

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

```
Number of Filters : 64
Kernel Size       : 3 x 3
Input Channels     : 3
Parameters/Filter : 28
Total Parameters   : 1792
```

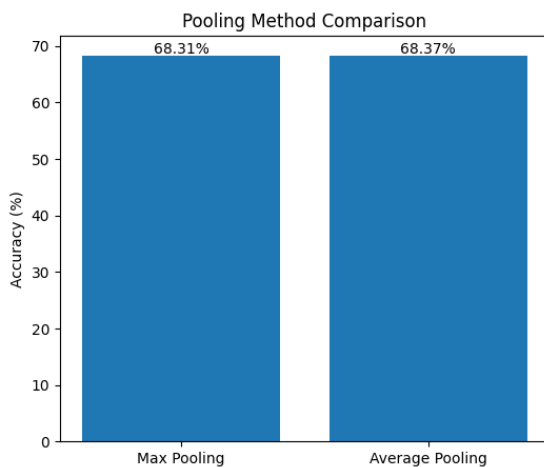
3. Compare ReLU and Sigmoid activation functions.



4. Replace Max Pooling with Average Pooling and compare the results.

Pooling Comparison	

Max Pooling Accuracy	: 68.31%
Average Pool Accuracy	: 68.37%



5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

```
64-Filter CNN Results
-----
Testing Accuracy : 66.18%
Training Time    : 3566.33 seconds
```

16 Filters vs 64 Filters

=====

16 Filters - Accuracy : 67.59%

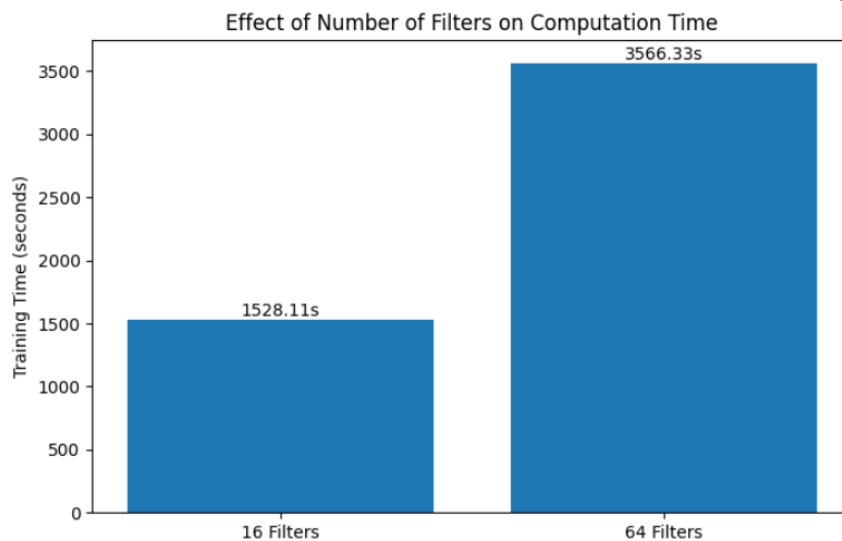
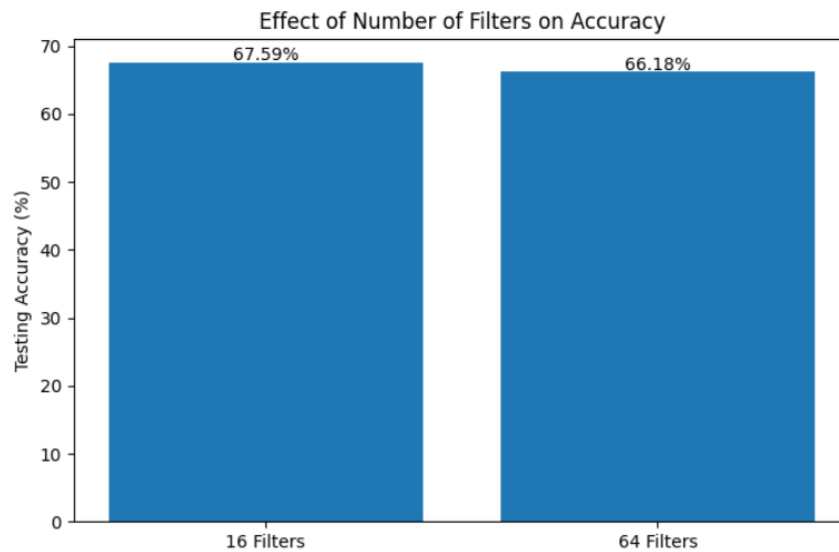
64 Filters - Accuracy : 66.18%

16 Filters - Time : 1528.11 seconds

64 Filters - Time : 3566.33 seconds

16 Filters - Parameters: 535,434

64 Filters - Parameters: 564,426



6. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

7. Discussion

Answer the following.

- a. Why is convolution preferred over fully connected layers for images?

Convolution is preferred because it can detect important features such as **edges, textures, shapes, and patterns** while looking at small regions of an image. It also uses the same filter across the image, so the number of parameters is much smaller than in a fully connected layer. This makes CNNs more efficient for image processing.

- b. How does stride affect the feature map size?

Stride determines how far the convolution filter moves across the image at each step. A **larger stride skips more pixels**, so the resulting feature map becomes smaller. For example, increasing the stride from 1 to 2 reduces the spatial dimensions of the feature map and also reduces computation.

- c. What is the role of padding?

Stride determines how far the convolution filter moves across the image at each step. A **larger stride skips more pixels**, so the resulting feature map becomes smaller. For example, increasing the stride from 1 to 2 reduces the spatial dimensions of the feature map and also reduces computation.

- d. Why is pooling used?

Pooling is used to **reduce the size of feature maps** while keeping the most important information. It reduces computation and memory usage and can also make the network less sensitive to small changes in the position of objects. Max pooling, for example, keeps the strongest value from each region.

- e. How do feature maps represent image characteristics?

Feature maps show the features detected by convolution filters at different locations in an image. In the early layers, they may detect simple things like **edges, corners, and colors**. As we move deeper into the CNN, the feature maps can represent more complex patterns such as textures, shapes, and parts of objects.

- f. Why do CNNs require fewer parameters than MLPs?

CNNs require fewer parameters mainly because they use **local connections and weight sharing**. A filter is applied to different parts of the image using the same set of weights. In contrast, an MLP connects every input pixel to every neuron, which creates a very large number of parameters. This makes CNNs much more suitable for large image data.

8. References

- a. Goodfellow et al., *Deep Learning*.
- b. Bishop, *Pattern Recognition and Machine Learning*.
- c. Haykin, *Neural Networks and Learning Machines*.
- d. TensorFlow Documentation.
- e. CIFAR-10 Dataset Documentation.

14. Conclusion

In this experiment, we implemented a **Convolutional Neural Network (CNN)** using TensorFlow/Keras for image classification on the **CIFAR-10 dataset**. We studied convolution, kernels, stride, padding, pooling, and feature maps. We also trained and evaluated the CNN using accuracy, precision, recall, F1-score, and a confusion matrix.

Overall, the experiment helped us understand how CNNs automatically learn useful image features and why they are effective for **computer vision and image classification tasks**